

UDESC

Departamento de Ciência da Computação

Teoria dos Grafos: Tarefa 1

**Representação Computacional de um grafo não
direcionado através de uma Lista de Adjacências**

Alunos: Heitor Linhares Caetano e Deivid Veras Lourenço

Professor: Gilmário Barbosa dos Santos

Maio
2026

Conteúdo

1	Objetivo do Trabalho	1
2	Solução fornecida e sobre o projeto	2
2.1	Leitura do CSV	2
2.2	Grau mínimo e máximo	3
2.3	Quantidade de laços e arestas múltiplas	3
2.4	Determinação de componentes conexos	4
3	Sobre o ambiente de desenvolvimento e compilação	5
4	Análise dos Resultados	6
5	Conclusões	7

1 Objetivo do Trabalho

A atividade consistiu em implementar um grafo através da representação computacional *Lista de Adjacências* e que é carregado a partir de uma base de dados no formato CSV.

A Lista de Adjacências é uma estrutura de dados composta por um vetor estático de Nós (unidade básica de uma lista encadeada). No nosso caso, cada índice do vetor representa um vértice diferente ($v_1, v_2, \dots$), e cada lista associada a um índice representa as adjacências do vértice correspondente.

Vértice	Adjacentes
1	2, 3, 5
2	1, 4
3	1
4	2, 5
5	1, 4

Figura 1: Representação de uma lista de adjacências. O vértice 1 é conectado a 2, 3 e 5.

Foram implementadas funções que fornecem o valor máximo e mínimo de um vértice, funções que determinam o tipo do grafo (simples ou multi-grafo), além da quantidade de laços e/ou arestas múltiplas que ocorrem nele e, como função principal, um procedimento que determina quantos componentes conexos existem no grafo, bem como sua distribuição e seus respectivos tamanhos.

2 Solução fornecida e sobre o projeto

O projeto foi administrado em um [repositório](#) do GitHub, e separado em arquivos diferentes para melhor organização do trabalho. Um arquivo para as funções referentes à Lista de Adjacências, outro referente a leitura e interpretação do arquivo CSV, outra para a interface de usuário e ainda um arquivo header, que possui as definições e protótipos usados nos outros.

A seguir vai uma explicação simples sobre cada exigência:

2.1 Leitura do CSV

Primeiramente, é importante ressaltar que o arquivo enviado não é exatamente um CSV, pois os valores ali estão separados por espaços, e não por vírgulas. Isso não é um grande problema, visto que alterar o código para suportar esse "SSV" seria trivial.

Mesmo assim, decidimos alterar o arquivo da base de dados a fim de transformá-lo em um CSV. Se tratou de uma tarefa simples, necessitando apenas os seguintes comandos em *Bash*:

```
$ tr ' ' ',' < teste2.csv > tmp.csv  
$ mv -f tmp.csv teste2.csv
```

Figura 2: O comando *tr* está trocando todas as aparições do caractere espaço por uma vírgula.

Antes de começarmos a ler os valores, precisamos determinar o tamanho do vetor estático que abrigará as adjacências. Para isso, foi criada uma função que corre por todos os números e acha o maior dentre eles. Esse número (+1) é então utilizado como tamanho do vetor, representando o maior índice existente no grafo.

Um possível problema dessa implementação é a eventualidade de não existir algum vértice entre 1 e o maior número, o que poderia significar que ele simplesmente não existe, mas aqui é interpretado como um vértice de grau nulo. Em uma conversa com o professor, o mesmo declarou nossa solução como válida.

Ao ler o arquivo, como estamos tratando de grafos não direcionados, para cada par de valores a, b precisamos adicionar um novo *nó* às listas de adjacências de ambos os vértices. No caso em que $a = b$, essa inserção é realizada apenas uma vez, a fim de evitar a duplicação de laços.

2.2 Grau mínimo e máximo

Aqui a implementação foi relativamente direta. Nos dois casos, todas as listas têm seus tamanhos contados através de um laço de repetição e o menor/maior número é retornado no final.

2.3 Quantidade de laços e arestas múltiplas

A existência de laços e/ou arestas múltiplas em um grafo confirma que este é um multigrafo. Existem várias abordagens possíveis para a contagem de valores repetidos, e a complexidade de cada uma foi levada em conta para a nossa escolha final.

Inicialmente, a ideia foi de fazer a contagem através de um vetor de frequência. Nessa estratégia, para cada vértice adjacente na lista, incrementa-se a posição correspondente em um vetor auxiliar. Ao final da leitura, valores maiores que 1 significam a presença de múltiplas arestas. Isso nos permite contabilizar duplicatas e laços de maneira linear.

Essa abordagem tem, para cada vértice, complexidade de tempo $O(g+V)$, onde g é o grau do vértice e V o número total de vértices do grafo. Isso porque um vetor de tamanho V era inicializado e percorrido em cada vértice, o que significa que a complexidade de espaço também era $O(V)$. Em grafos esparsos (como o analisado), isso significa que ocorre um desperdício de memória, pois grande parte do vetor nem chega a ser usada.

Como o procedimento é executado para cada vértice, a complexidade total da função era $O(V^2 + E)$, onde E é o número de arestas. Assim, o desempenho dependia fortemente da quantidade total de vértices do grafo, e não apenas das arestas existentes.

A segunda abordagem, que acabou sendo escolhida, consiste em copiar os valores da lista a um vetor auxiliar e, em seguida, ordená-lo. Após a ordenação, basta percorrer o vetor uma única vez verificando elementos consecutivos iguais para identificar arestas múltiplas.

Essa implementação possui complexidade $O(g \log g)$ para cada vértice. Mesmo sendo *assintoticamente* menos eficiente do que a estratégia com vetor de frequência, essa solução tira a dependência do número total de vértices, sendo mais "bem comportada" para grafos dos mais diversos tipos.

Além disso, a complexidade espacial passa a ser $O(g)$, já que o vetor auxiliar possui tamanho proporcional apenas ao grau do vértice analisado. Em grafos esparsos, isso reduz significativamente o consumo de memória. Com o procedimento sendo executado para cada vértice, a complexidade temporal total da função é de $O(\sum_{i=1}^V g_i \log g_i)$, com limite superior de $O(E \log V)$.

Vale a pena ressaltar que os laços recebem tratamento diferente dos outros vértices repetidos. Enquanto arestas múltiplas aparecem duas vezes na lista de adjacência, o que faz com que se exija uma divisão por dois no final, laços são armazenados apenas uma vez, sendo contabilizados separadamente.

2.4 Determinação de componentes conexos

3 Sobre o ambiente de desenvolvimento e compilação

Como explicado anteriormente, um repositório de código-fonte foi instalado no GitHub para uma melhor colaboração.

Como sistema operacional, foram utilizados tanto o Windows (Deivid) quanto GNU/Linux (Heitor). Mesmo com ambientes diferentes, miramos em ter uma *codebase* compatível, sem usar funções não portáteis, como *system()*.

Sobre os ambientes de desenvolvimento, Deivid usou o [Visual Studio Code](#), enquanto Heitor usou o editor de texto [Helix](#). Como este não usou uma IDE completa, algumas ferramentas de linha de comando foram usadas para emular funcionalidades comuns em outros ambientes, especificamente [cproto](#) para a geração automática de protótipos e [astyle](#) como formatador de código.

Como as demais ferramentas, também foi utilizada a linha de comando para compilar o projeto, em que foi adotado o compilador de C do *GNU (GCC)*.

O comando para compilar é:

```
$ gcc ls_adj.c csv.c main.c -o grafo
```

e para iniciar o programa, simplesmente execute:

```
$ ./grafo
```

4 Análise dos Resultados

5 Conclusões